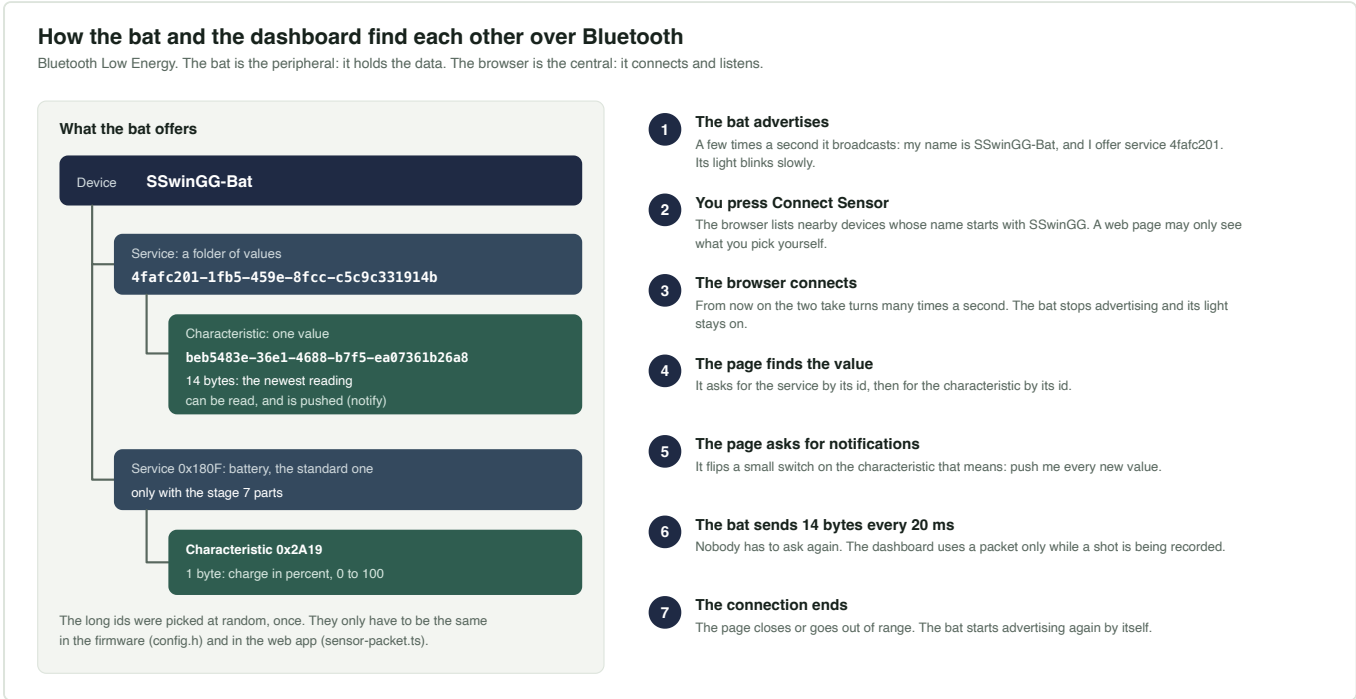
Stage 5: send the readings over Bluetooth

You add: nothing. Still on the USB cable.

You learn: how a Bluetooth Low Energy device is found, connected to and listened to, and what the 14-byte packet looks like.



Steps

1. Lay the board still. Open 05_ble_stream.ino and upload. The light blinks slowly: it is waiting for a connection.
2. On the computer, serve the test page. It has to come from `localhost`, because browsers block Bluetooth for pages opened as files. In a terminal, from this kit's folder:

   ```
   cd tools
   python3 -m http.server 8000
   ```

3. Open `http://localhost:8000/ble_test.html` in **Chrome or Edge**.
4. Press **Connect to the bat** and choose `SSwinGG-Bat`.
5. Swing the board.

No computer nearby? The phone app nRF Connect also works: scan, connect to `SSwinGG-Bat`, open the service, and switch notifications on for its one value.

What you should see

On the board, the light goes steady. In the Serial Monitor:

```
===== Stage 5: Bluetooth stream =====
BMI160 found at 0x68
Keep the board still for one second...
Bluetooth is on. Look for "SSwinGG-Bat".
waiting    sent 0/s  failed reads 0  last packet: 2F 00 AB FF 0E 10 01 00 00 00 00 00 00
connected  sent 12/s  failed reads 0  last packet: 32 00 AF FF 13 10 00 00 FF FF 01 00 00 00
connected  sent 50/s  failed reads 0  last packet: 35 05 72 03 12 10 40 FD 06 0B 4C 01 00 00
connected  sent 50/s  failed reads 0  last packet: 2F 00 AB FF 0E 10 02 00 01 00 01 00 00 00
Disconnected. Advertising again.
waiting    sent 13/s  failed reads 0  last packet: 34 00 AF FF 11 10 02 00 00 00 01 00 00 00
```

On the test page:

- about 50 readings per second
- all four checks pass while the board lies still
- the table "The last packet, taken apart" shows the same bytes as the Serial Monitor, split into seven numbers
- a swing makes the bat speed rise and the chart draw a peak

The stage is passed when the page shows 50 readings a second and a speed of 0 at rest.

"Try with a simulated bat" on the page shows what a working connection looks like, with numbers the page makes up itself.

What is going on

The 14-byte packet

What the bat sends 50 times a second, and how the dashboard reads it. The example was taken during a swing.

byte 0	byte 1	byte 2	byte 3	byte 4	byte 5	byte 6	byte 7	byte 8	byte 9	byte 10	byte 11	byte 12	byte 13
35	05	72	03	12	10	40	FD	06	0B	4C	01	00	00

ax	ay	az	gx	gy	gz	temperature
0x0535	0x0372	0x1012	0xFD40	0x0B06	0x014C	0x0000
= 1333 counts	= 882 counts	= 4114 counts	= -704 counts	= 2822 counts	= 332 counts	= 0 counts
/ 4096	/ 4096	/ 4096	/ 16.4	/ 16.4	/ 16.4	not used by the dashboard
+0.325 g	**+0.215 g**	**+1.004 g**	**-42.9 deg/s**	**+172.1 deg/s**	**+20.2 deg/s**	

What the dashboard does with these six numbers

Total rotation = square root of gx squared + gy squared + gz squared = 178.5 degrees per second
Bat speed = total rotation x pi / 180 x 0.85 m x 3.6 = 9.5 km/h
Impact = square root of ax squared + ay squared + az squared = 1.08 g
Shot direction = leg side if gz is above +1, off side if below -1, otherwise straight = leg side

Low byte first. The number 0x0B06 travels as 06 then 0B. Both ends must agree on this, and on the order of the seven numbers.
Signed. Each number runs from -32768 to +32767. A value with its top bit set is negative: 0xFD40 is -704.
Why counts and not units. Seven whole numbers fit in 14 bytes. The same values as text, as stage 9 sends them over WiFi, need about 110 bytes.
The scale, 4096 and 16.4, comes from the sensor's ranges of 8 g and 2000 degrees per second. Change a range and both ends have to change.

- **Advertising.** `BLEDevice::startAdvertising()` makes the board broadcast its name and the id of its service a few times a second.
- **Server, service, characteristic.** The board holds the data, so it is the server. The sketch creates one service, a folder, and in it one characteristic, a value. Both are named by a long id that the dashboard asks for by heart.
- **Notify.** Every 20 milliseconds the sketch puts a new 14-byte value into the characteristic and calls `notify()`. The connected browser receives it without asking.

- **The packet.** Seven 16-bit numbers, low byte first: ax, ay, az, gx, gy, gz, temperature. `swing_packet.h` builds it. The browser divides the first three by 4096 and the next three by 16.4.
- **Callbacks.** `onConnect` and `onDisconnect` are called by the Bluetooth code when someone arrives or leaves. They only set a flag, and `loop()` acts on it.
- **Two radios, one chip.** Bluetooth is built into the ESP32-C3. No part was added for this stage.

Try this

- Compare the "last packet" line in the Serial Monitor with the table on the test page. Work one value out by hand.
- Change `SAMPLE_PERIOD_MS` to `100` and upload. The page now shows 10 readings a second and its first check fails. Put it back to 20.
- Close the test page. The board prints "Disconnected. Advertising again." and the light blinks again.

If it does not work

See Bluetooth.

Next: stage 6.